

CAMPUS PRINT HUB — INCIDENT & ARCHITECTURE REPORT

Subject: Technical Audit of Agent Offline Issues & Cloudflare Tunnel Dual-Tunnel Architecture

1. OVERVIEW

During the deployment of the Campus Print Agent on the physical Shop PC, the agent successfully executed local printer scans and established registration, but continued to show as 'OFFLINE' in the Admin Console. This report outlines the technical and logical reasons for this behavior and explains the architectural changes made to solve the issue.

2. PROBLEM A: THE DOUBLE PROXY & VITE DEV SERVER TIMEOUTS

Logical Explanation:

The initial tunnel setup used a single Cloudflare Tunnel pointing to port 3000 (the Vite Dev Server). The Vite Dev Server was configured to proxy all API requests (under /api) to the Express Backend running on port 3001.

Technical Impact:

1. Vite's built-in dev proxy is designed for standard, short-lived HTTP requests (like fetching JSON data or assets).
2. The print agent relies on two long-running persistent connections:
 - Heartbeat requests that must respond instantly to maintain alive status.
 - Server-Sent Events (SSE) stream via ``/api/jobs/stream`` for real-time job push.
3. Vite's proxy periodically terminated these long-standing TCP streams due to default activity timeouts. This caused the agent to repeatedly disconnect, drop offline, and fail to receive print commands in real time.

The Dual-Tunnel Solution:

To solve this, we split the front-end browser traffic and the back-end agent traffic into two separate Cloudflare Tunnels:

- Tunnel 1 (Port 3000): Serves only the browser/frontend (HTML, React, CSS). This handles user interactions.
- Tunnel 2 (Port 3001): Directly exposes the Express Backend. The print agent's config.json is configured to point directly to this backend tunnel.

By pointing the print agent directly to port 3001, we bypassed the Vite proxy layer entirely, allowing SSE and heartbeats to stay open indefinitely without timeout.

3. PROBLEM B: AUTHORIZATION ROUTE BYPASS IN FRONTEND

Logical Explanation:

The server protects the printer settings endpoint ``/api/printer/settings`` using the ``requireAdmin`` middleware to prevent unauthorized access to hardware configurations.

Technical Impact:

1. The React app's ``App.tsx`` file fetches these settings inside ``fetchPrinterSettings`` to render the agent status badge.
2. However, this fetch request was made without sending the 'Authorization' header.
3. As a result, the server rejected the React application's request with a HTTP 401 Unauthorized code.
4. Because the request failed silently in the background, the UI's state was never updated with the real agent status and stayed hardcoded to the default state: 'offline'.

The Code Fix:

We updated `App.tsx` to retrieve the active shop admin's token from sessionStorage and append it to the request headers:

```
headers: { 'Authorization': `Bearer ${token}` }
```

4. PROBLEM C: STATE PROPERTY MISMATCH**Technical Impact:**

The frontend was reading `settings.agentLastHeartbeat` to display the last updated time. However, the backend returns this property as `lastHeartbeat`. We aligned the property key so the timestamp updates dynamically.